

What I Built:

Dense $\rightarrow$ Vanilla: $N \times N$

Sliding $\rightarrow$ each sees the last W

Sliding + global $\rightarrow$ same window + first G tokens act as hubs

{ All have causal masking
 & tested across len: 8 to 8192, 116k param }

Sparsity Patterns:

* token @ posn 500 CANNOT see anything b4 469.

So info can only propagate across window using global tokens are kind of fixing

So 100 can reach 500,

in 2 hops 100 - global - 500

So a connectivity is just 2 hops

it becomes a connectivity issue & not
a volume argument

NaN handling:

It happens when we combine both on an
early & you get a query row where
every single masked.

So what I did detect fully masked rows
before softmax, replace their scores with 0
& 0 the output over averaging.

Boon py shows this scenario
↳ shows NaN
without guard

The Benchmark:

* CPU was used: CPU used, 2.10.0 (Torch)

O/P : (kn)	dense (ms)	sliding (ms)	global (ms)
512	1.64	5.61	6.62
1024	5.0	14.91	15.07
2048	22.24	39.41	45.48
4096	93.45	127.08	144.08

These implementations & mask (masked are still computed)

Professional stuff → Don't

So what i built is just correctness

&

Dense ↑ quadratically while
sparse → linear

Quality Eval:

model: 116k params, 1500 iter, Global⁴
train loss val loss

dense : 2.3756 2.3853

sliding : 2.3198 2.3496

global : 2.3291 2.3517

Analysis:

sparse's > dense

Q SLIDING → BEST:

why counter intuitive observation:

dense has access to long-range correlation
that sparse doesn't

dense can latch onto spurious patterns
& overfit to them while sliding forces

the model focuses on predictive patterns

A bigger model "could" fit long range dependencies